

Architecture et Algorithmes de la Génération Procédurale de Cartes dans la Série Civilization

La génération procédurale de cartes dans la franchise *Sid Meier's Civilization* constitue un modèle d'ingénierie logicielle combinant géométrie computationnelle, simulation géophysique et algorithmes d'équilibrage de jeu compétitif¹. L'objectif fondamental du système est de créer des mondes visuellement crédibles, biologiquement cohérents et parfaitement équilibrés pour la compétition entre plusieurs civilisations aux besoins asymétriques¹.

Architecture Globale du Pipeline de Génération Procédurale

L'architecture logicielle de génération de cartes dans la franchise a connu une évolution majeure entre *Civilization IV* et *Civilization V*³. Historiquement codée en C++ au sein du moteur principal via des classes dédiées telles que `CvSiteEvaluatorForStart`, la logique de génération a été presque intégralement déportée vers une couche de scripts Lua à partir de *Civilization V* et conservée dans *Civilization VI*³. Ce choix architectural offre une extensibilité totale pour le modding, bien qu'il nécessite la recreation en script de fonctions d'évaluation géométrique et algorithmique complexes³.

Le processus complet suit une exécution séquentielle stricte où chaque étape dépend des structures de données générées par la précédente⁴. Les masses terrestres fondamentales sont d'abord délimitées avant que les gradients climatiques ne déterminent les biomes, suivis par le tracé des réseaux hydrographiques et enfin par la segmentation territoriale dédiée à l'équilibrage des joueurs¹.

Étape du Pipeline	Fichiers et Scripts Clés	Algorithmes et Principes Associés	Données Formées en Sortie
1. Altimétrie & Masses Terrestres	FractalWorld.lua, Continents.lua [cite: 4, 6]	Bruit fractal (Perlin/Simplex), Pavage de Voronoï, Tectonique ²	Masque Continental (Terre / Eau), Cartographie des Rifts ⁷
2. Climat & Biomes	TerrainGenerator.lua [cite: 4, 9]	Gradients latitudinaux, Équations de température et	Types de terrain de base (Prairie, Plaine, Désert, Toundra, Neige) ¹

		d'humidité ¹	
3. Altitudes Secondaires & Reliefs	MountainsCliffs.lua, FeatureGenerator.lua [cite: 4, 9]	Masques de bruit secondaire, Érosion continentale ²	Montagnes, Collines, Volcans, Lignes de falaises ²
4. Hydrographie (Rivières/Lacs)	RiverGenerator.lua [cite: 6]	Recherche de chemin sur graphe dual d'arêtes hex (A* / Dijkstra) ¹²	Rivières sur arêtes d'hexagones, Lacs ¹¹
5. Couverture Végétale & Micro-biomes	FeatureGenerator.lua [cite: 4, 9]	Évaluation des conditions locales (latitude, humidité, relief) ⁵	Forêts, Jungles, Marais, Oasis, Glaces de mer ⁵
6. Segmentation & Fertilité	AssignStartingPlots.lua [cite: 1, 4, 5]	Algorithme de Voronoï sur métrique de surface, Scoring de fertilité ⁵	Territoires régionaux, Matrice de valeurs de cases ¹
7. Placement des Capitales	AssignStartingPlots.lua [cite: 1, 5]	Biais d'apparition (Start Biases), Contraintes de distances relatives ¹	Coordonnées des emplacements de départ des joueurs ¹
8. Équilibrage & Ressources	AssignStartingPlots.lua [cite: 4, 5]	Algorithme de normalisation des départs, Distribution de monopsons ⁵	Ressources stratégiques, de luxe et de bonus ⁵

Génération du Relief et Topographie : Du Bruit Fractal aux Diagrammes de Voronoï

L'Approche par Bruit Fractal (Civilization V et VI)

Dans *Civilization V* et *VI*, la topographie repose sur la superposition d'octaves de bruit fractal². La classe scriptée *FractalWorld* génère une grille de valeurs continues comprises entre 0 et 100⁶. Ces valeurs sont ensuite seuillées en fonction des paramètres sélectionnés par le joueur, comme le niveau de la mer, pour différencier l'océan profond, les eaux côtières et les terres émergées². Pour éviter la formation systématique d'un unique supercontinent informe, le script

applique des algorithmes de rifts tectoniques⁷. Des lignes de fracture sont tracées dynamiquement à travers la grille fractale, abaissant artificiellement l'altitude le long de ces axes afin de forcer la création d'océans séparateurs et de former des continents distincts⁷.

La Révolution des Plaques Tectoniques et Voronoï (Civilization VII)

Civilization VII a introduit un changement de paradigme fondamental en abandonnant la génération fractale pure au profit d'un système multi-résolution fondé sur les diagrammes de Voronoï et la simulation de plaques tectoniques². Le processus débute par un semis initial à basse résolution, où un ensemble restreint de points germes est distribué de manière pseudo-aléatoire sur l'espace de la carte². Une cellularisation de Voronoï divise la carte en polygonations s'apparentant à du verre brisé, et certaines cellules sont choisies pour servir de noyaux initiaux aux plaques tectoniques². Chaque plaque s'étend cellule par cellule selon des règles de configuration géométrique jusqu'à couvrir l'intégralité du globe, recevant un vecteur de déplacement qui définit la dynamique de la croûte terrestre².

Un second semis de points, beaucoup plus dense, est ensuite projeté sur la grille afin de générer un pavage de Voronoï secondaire à haute définition superposé aux plaques². Les masses continentales principales croissent sur ce pavage haute résolution selon des contraintes géophysiques et des règles de gameplay². Elles s'étendent préférentiellement le long des zones de collision de plaques, évitent les régions polaires, respectent des bandes latitudinales et maintiennent une séparation stricte entre les terres d'origine (*Homelands*) et les terres lointaines (*Distant Lands*) par des océans deep². Des règles géomorphologiques secondaires génèrent enfin les îles, appliquent une érosion côtière et élèvent les chaînes de montagnes ou arcs volcaniques aux zones de convergence des plaques avant de plaquer le relief vectoriel sur la grille hexagonale².

Mathématiques de la Grille Hexagonale

La manipulation des cases sur la carte requiert un système de coordonnées rigoureux. Le système canonique employé est celui des coordonnées cubiques modélisées par un triplet (q, r, s) restreint par la contrainte planaire¹² :

$$q + r + s = 0$$

Ce système garantit une symétrie parfaite selon les trois axes géométriques de la grille hexagonale¹². La distance hexagonale exacte entre deux cases $A(q_1, r_1, s_1)$ et $B(q_2, r_2, s_2)$ s'exprime par l'équation¹² :

$$d(A, B) = \max(|q_1 - q_2|, |r_1 - r_2|, |s_1 - s_2|)$$

Cette métrique est privilégiée dans l'ensemble des algorithmes du jeu pour évaluer les distances de sécurité, l'accès aux ressources et l'influence territoriale⁵.

Simulation Climatique, Hydrographie et Placement des Biomes

Calcul des Terrains et Gradient Latitudinal

Une fois le relief posé, le script TerrainGenerator.lua détermine le type de terrain de base de chaque case⁴. Le facteur primaire est la latitude de la case, modélisée par sa distance relative à

l'équateur et aux pôles¹. La température théorique T décroît proportionnellement à la valeur absolue de la latitude, modulée par l'altitude (les collines et montagnes subissent un refroidissement)⁵.

Des seuils de température déterminent le terrain fondamental¹ :

- **Neige** : Zones polaires à températures extrêmement basses¹¹.
- **Toundra** : Sub-polaire à températures basses⁵.
- **Prairie, Plaine, Désert** : Zones tempérées à équatoriales, différenciées selon le taux d'humidité¹.

Simulation des Précipitations et Effets d'Ombre Pluviométrique

L'humidité est modélisée soit par une carte de bruit secondaire, soit par une simulation de circulation atmosphérique simplifiée (comme dans le script communautaire *Communitas*)¹⁰. Dans ces modèles avancés, des vents dominants virtuels transportent l'humidité depuis les cases d'océan¹⁰. Lorsque cette masse d'air rencontre une chaîne de montagnes, l'humidité précipite sur le versant au vent, tandis que le versant sous le vent subit un assèchement drastique (effet d'ombre pluviométrique), générant des bandes désertiques de manière physiquement cohérente¹⁰.

Modélisation des Cours d'Eau sur les Arêtes Hexagonales

Contrairement aux tuiles de terrain qui occupent le centre des hexagones, les rivières dans *Civilization* possèdent la particularité topologique de circuler sur les arêtes qui séparent les hexagones¹². L'algorithme de génération de rivières (RiverGenerator.lua) fonctionne en construisant un graphe dual où les sommets sont les points d'intersection des hexagones, et les arêtes du graphe correspondent aux limites des cases⁶.

Les sources de rivières sont d'abord initiées sur des sommets adjacents à des reliefs élevés (montagnes ou collines) possédant une humidité minimale⁵. La rivière s'écoule ensuite de sommet en sommet en cherchant la pente la plus forte vers le niveau de la mer. Pour éviter que les rivières ne forment des boucles infinies ou ne coupent des masses terrestres de manière incohérente, un algorithme de recherche de chemin de type A* ou Dijkstra est exécuté sur le graphe d'arêtes¹². Les arêtes déjà occupées par un cours d'eau ou bordant immédiatement un océan se voient attribuer un coût de traversée infini¹². Si une rivière atteint une cuvette dépressionnaire sans accès à la mer, l'écoulement s'arrête et la case adjacente la plus basse est convertie en lac¹¹.

Placement de la Végétation et des Caractéristiques (FeatureGenerator.lua)

Les caractéristiques de surface (**Forêt, Jungle/Forêt tropicale, Marais, Oasis**) sont appliquées lors d'une passe ultérieure contrôlée par FeatureGenerator.lua⁴. Elles dépendent du croisement entre le terrain sous-jacent et le niveau d'humidité⁵. La jungle est restreinte aux latitudes équatoriales sur des terrains de prairie ou de plaine à très haute humidité¹. La forêt apparaît dans les zones tempérées et froides sur prairies, plaines ou toundras¹. Les marais se développent sur des cases de prairie plates et basses adjacentes à de l'eau douce⁵. Enfin, les oasis se positionnent exclusivement sur de petites cuvettes de désert plat non bordées par d'autres sources d'eau douce⁵.

Algorithmique des Positions de Départ et Équilibrage des Ressources (AssignStartingPlots.lua)

Le fichier AssignStartingPlots.lua résout un problème d'optimisation sous contraintes multiples : placer chaque joueur sur une case qui maximise le potentiel stratégique propre à sa civilisation, tout en garantissant un niveau de fertilité global équivalent entre tous les concurrents¹.

Le flux d'exécution de AssignStartingPlots.lua suit une progression rigoureuse en cinq phases clés :

1. **Partitionnement Régional** : La carte est découpée en N secteurs équivalents au nombre de civilisations en jeu¹.
2. **Évaluation de la Fertilité** : Un scoring approfondi évalue les rendements en nourriture et production sur trois anneaux concentriques autour des cases candidates⁵.
3. **Appariement des Biais** : Les civilisations sont assignées aux régions selon leurs affinités géographiques uniques (ex. désert pour l'Arabie, forêts pour les Celtes)¹.
4. **Normalisation des Départs** : Les départs défavorisés reçoivent des injections compensatoires de ressources stratégiques et de bonus⁵.
5. **Distribution des Monopoles** : Les ressources de luxe sont réparties par paires régionales et monopsons pour stimuler les échanges commerciaux¹.

Partitionnement Régional de la Carte

Avant de placer la moindre unité, l'algorithme divise l'intégralité de la masse terrestre en un nombre de régions territoriales égal au nombre de civilisations en jeu¹. Ce découpage utilise un algorithme de croissance de régions de type Voronoï pondéré². La métrique de taille d'une région n'est pas purement géométrique : elle intègre la densité de cases exploitables⁵. Une région composée principalement de toundra recevra une surface géométrique plus grande pour compenser la faible valeur de ses cases⁵.

Calcul du Score de Fertilité (MeasureStartPlacementFertilityOfPlot)

L'algorithme évalue la valeur globale de chaque case candidate pour devenir un emplacement de capitale⁵. Pour une case donnée, le script calcule un score de fertilité en analysant les cases contenues dans les premier, deuxième et troisième anneaux d'exploitation urbaine⁵.

La fonction de fertilité de base s'appuie sur la formule pondérée suivante :

$$\text{Fertilité}(P) = \sum_{k=1}^3 w_k \cdot \left(\sum_{t \in \text{Anneau}(k)} \left(Y_{\text{Nourriture}}(t) \cdot \alpha + Y_{\text{Production}}(t) \cdot \beta + B_{\text{EauDouce}}(t) - P_{\text{Inutilisable}}(t) \right) \right)$$

Où :

- w_k représente le poids géométrique de l'anneau ($w_1 = 1.0$, $w_2 = 0.6$, $w_3 = 0.3$).
- $Y_{\text{Nourriture}}(t)$ et $Y_{\text{Production}}(t)$ sont les rendements de base théoriques de la case t au tour 0 (ex. Prairie = 2 Nourriture, Plaine = 1 Nourriture / 1 Production, Colline = 2 Production)⁵.
- $B_{\text{EauDouce}}(t)$ est un bonus accordé aux cases bordées par une rivière, un lac ou une oasis⁵.
- $P_{\text{Inutilisable}}(t)$ est un malus sévère appliqué aux cases non exploitables (Montagnes, Glace, Toundra plate nue, Désert plat)⁵.

Les cases accumulant un nombre trop élevé de tuiles improductives (par exemple 9 cases ou plus sans rendement dans le rayon) sont immédiatement rejetées et marquées comme invalides¹¹.

Biais d'Apparition des Civilisations (Start Biases)

Chaque civilisation possède des règles d'affinité définies dans ses données de jeu¹.

L'algorithme apparie les régions calculées avec les joueurs en cherchant la meilleure adéquation¹ :

- **Biais de Terrain** : L'Égypte priorise les régions désertiques, les Celtes les zones boisées, le Brésil la jungle¹.
- **Biais Côtier** : Venise ou l'Angleterre requièrent impérativement un emplacement situé sur une case de côte¹.
- **Attribution Sans Biais** : Les civilisations ne possédant aucun biais particulier sont distribuées aléatoirement sur les régions restantes¹.

Si la carte générée ne contient aucune zone correspondant au biais d'une civilisation, le biais est annulé et la civilisation est placée sur le meilleur emplacement générique disponible¹.

Procédure de Normalisation et Distribution des Ressources

Pour pallier les inégalités naturelles de la génération procédurale, AssignStartingPlots.lua exécute la fonction NormalizeStartLocation avant la distribution générale des ressources sur le reste du globe⁵. Le script mesure la fertilité effective cumulée autour du point de départ retenu⁵. Si l'option d'équilibrage stratégique est activée, le système force le placement d'un gisement mineur de Fer et de Chevaux à une distance stricte de 4 à 6 cases de la capitale⁵. Si le score

de nourriture du premier anneau est insuffisant, le script transforme des cases neutres en ressources de bonus de nourriture (**Bison, Cerf, Blé, Vaches, Moutons, Poisson**) ou génère une **Oasis**⁵. En cas de manque de rendement industriel sur un départ composé ultra-majoritairement de prairies plates, le script force l'apparition de cases de **Pierre** sur les tuiles plates⁵.

Répartition Régionale des Ressources de Luxe

Pour favoriser le commerce et la diplomatie, le système de ressources de luxe utilise un algorithme de distribution par monopsons régionaux¹ :

Catégorie de Luxe	Règle de Placement et Distribution	Impact sur le Gameplay
Luxe de Monopole	Exclusif à une région spécifique. Apparaît en très grande abondance dans cette zone et nulle part ailleurs sur le continent ¹ .	Force les joueurs à commercer avec le possesseur du monopole pour obtenir cette ressource ¹ .
Luxe Secondaire	Placé dans le rayon de 3 cases autour de la capitale de chaque joueur ¹ .	Garantit une seconde source immédiate de bonheur/ressources en début de partie ¹ .
Luxe Unique Sub-régional	Généré dans un rayon de 8 cases autour de la capitale ¹ .	Encourage l'expansion territoriale rapide et la fondation de secondes villes ¹ .
Luxe des Cités-États	2 à 4 types de luxe sont strictement exclus des territoires des joueurs majeurs et attribués uniquement aux Cités-États ¹⁶ .	Incite à la diplomatie ou à la conquête des Cités-États pour obtenir des luxes uniques ¹⁶ .

Guide d'Implémentation Technique pour un Moteur Indépendant

Pour un développeur souhaitant reproduire localement un moteur de carte procédurale de type *Civilization* (sous Unity, Unreal Engine ou un moteur C++/Rust propriétaire), il est recommandé de découpler le système en sous-modules fonctionnels interconnectés.

Architecture Logicielle Découpée

L'architecture du générateur se décompose en six modules indépendants s'exécutant de manière séquentielle :

1. **Topology Module** : Gère la grille hexagonale en coordonnées cubiques (q, r, s) et fournit les calculs de distances ou de voisinage¹².
2. **Noise & Tectonic Engine** : Génère le masque d'altitude de base via des voronoïs multi-résolutions ou des bruits fractals².
3. **Climate & Biome Resolver** : Calcule les températures latitudinales et applique les précipitations pour assigner les terrains de base¹.
4. **Graph River Generator** : Exécute un algorithme A* sur le graphe dual des arêtes d'hexagones pour tracer les cours d'eau¹².
5. **Fertility & Region Divider** : Partitionne la carte en régions équitables et évalue le scoring des cases sur 3 anneaux².
6. **Start Location & Resource Normalizer** : Assigne les joueurs selon leurs biais et injecte des ressources compensatoires¹.

Directives d'Implémentation Algorithmique

1. Abstraction de la Grille Hexagonale

Utiliser un système d'axes cubiques (q, r, s) sous forme de structures de données immutables¹². Implémenter les tables d'adjacence pour les 6 directions cardinales d'un hexagone¹⁷ :

```
C++
struct CubeCoordinate {
    int q, r, s;
    CubeCoordinate(int q_, int r_, int s_) : q(q_), r(r_), s(s_) {}
};

const CubeCoordinate CUBE_DIRECTIONS[6] = {
    CubeCoordinate(+1, 0, -1), CubeCoordinate(+1, -1, 0), CubeCoordinate(0, -1, +1),
    CubeCoordinate(-1, 0, +1), CubeCoordinate(-1, +1, 0), CubeCoordinate(0, +1, -1)
};
```

2. Traceur de Rivières sur le Graphe Dual

Créer une structure de données parallèle représentant les sommets (coins) des hexagones¹². Chaque sommet possède 3 sommets voisins interconnectés par des arêtes¹⁴.

- Utiliser un algorithme de recherche du plus court chemin A* dont la fonction heuristique

H privilégie la baisse constante de l'altitude géologique¹².

- Appliquer un coût prohibitif sur les sommets partageant une arête déjà marquée par une rivière pour éviter les chevauchements parallèles¹².

3. Algorithme d'Évaluation de Fertilité

Pour implémenter localement la fonction de scoring de `AssignStartingPlots.lua`, définir une matrice d'évaluation des cases basée sur les types de terrain et de ressources⁵ :

Python

```
def calculate_plot_fertility(candidate_plot, grid_map):
    total_fertility = 0.0
    ring_weights = [1.0, 0.6, 0.3] # Anneaux 1, 2, et 3

    for ring_idx, weight in enumerate(ring_weights):
        ring_plots = grid_map.get_ring(candidate_plot, ring_index=ring_idx + 1)
        for plot in ring_plots:
            if plot.is_mountain or plot.is_ice:
                total_fertility -= 10.0
            else:
                food_yield = plot.base_food + plot.feature_food
                prod_yield = plot.base_production + plot.feature_production
                fresh_water_bonus = 2.0 if plot.has_fresh_water else 0.0

                plot_score = (food_yield * 2.0) + (prod_yield * 1.5) + fresh_water_bonus
                total_fertility += weight * plot_score

    return total_fertility
```

4. Moteur de Normalisation des Départs

Afin d'éviter qu'un joueur humain ou une IA n'obtienne un départ inviable, implémenter une boucle de rattrapage⁵ :

- Si la fertilité totale de l'emplacement sélectionné est inférieure à la moyenne des autres joueurs de plus de 15%, lancer une passe de correction⁵.
- Parcourir les cases du rayon 1 et 2 et transformer les prairies arides en ressources de bonus (Blé, Bétail) jusqu'à ce que la fertilité équivale à la moyenne du serveur ou de la partie⁵.

Synthèse et Directives de Conception

La génération procédurale dans la série *Civilization* démontre qu'une carte de jeu réussie ne

peut résulter de la seule application de fonctions de bruit mathématique². La qualité du gameplay repose sur l'interaction entre deux couches distinctes : la couche géophysique et la couche d'équilibrage économique¹.

La couche géophysique génère la vraisemblance visuelle et la variété géographique en simulant des lois naturelles comme la tectonique de Voronoï, les gradients thermiques latitudinaux, les ombres pluviométriques et l'écoulement hydrographique sur graphe dual². La couche d'équilibrage économique fait abstraction des vérités géologiques pures pour corriger la carte de manière déterministe via la segmentation en régions, le calcul de fertilité multi-anneaux et la normalisation dynamique des ressources¹. Pour tout projet indépendant s'inspirant de ces principes, la clef réside dans la primauté de la couche d'équilibrage sur la couche géophysique : la carte doit d'abord servir les impératifs du gameplay et de l'équité compétitive avant de chercher la pureté architecturale de sa génération procédurale².

Sources des citations

1. Mechanics of starting locations : r/civ - Reddit, https://www.reddit.com/r/civ/comments/1kiesj/mechanics_of_starting_locations/
2. Improved Map Generation | Civilization VII, <https://civilization.2k.com/civ-vii/game-guide/gameplay/map-generation/>
3. AssignStartingPlots - An Exercise in Murder | CivFanatics Forums, <https://forums.civfanatics.com/threads/assignstartingplots-an-exercise-in-murder.540234/>
4. Steam 창작마당::Map Generation Fixing, <https://steamcommunity.com/sharedfiles/filedetails/?l=koreana&id=2337081565&searchtext=map+script>
5. AssignStartingPlots defaults for VP | CivFanatics Forums, <https://forums.civfanatics.com/threads/assignstartingplots-defaults-for-vp.662444/>
6. [BNW] - Adding a map size | CivFanatics Forums, <https://forums.civfanatics.com/threads/adding-a-map-size.677160/>
7. Map script request: blend of Continents and Fractal, <https://forums.civfanatics.com/threads/map-script-request-blend-of-continents-and-fractal.654286/>
8. Adding more islands [LUA map script] - CivFanatics Forums, <https://forums.civfanatics.com/threads/adding-more-islands-lua-map-script.570891/>
9. Procedural map generation | CivFanatics Forums, <https://forums.civfanatics.com/threads/procedural-map-generation.538717/>
10. On communitas map | Page 3 | CivFanatics Forums, <https://forums.civfanatics.com/threads/on-communitas-map.648143/page-3>
11. Starting Plot Assigner Tweaks : r/nqmod - Reddit, https://www.reddit.com/r/nqmod/comments/39f1sb/starting_plot_assigner_tweaks/
12. Pathfinding on a hexagonal grid - A* Algorithm - The Knights of U, <https://theknightsofu.com/pathfinding-on-a-hexagonal-grid-a-algorithm-2/>
13. Pathfinding in hex grid based game,

- <https://gamedev.stackexchange.com/questions/62546/pathfinding-in-hex-grid-based-game>
14. Implementing Path-Finding On A Hexagonal Board - Marmelab,
<https://marmelab.com/blog/2022/03/22/implementing-path-finding-on-a-hexagonal-board.html>
 15. Caps Lock [obsolete] - CivFanatics Forums,
<https://forums.civfanatics.com/threads/caps-lock-obsolete.624260/>
 16. Is there any logic to iron and coal deposits? : r/civ5 - Reddit,
https://www.reddit.com/r/civ5/comments/ye5l2k/is_there_any_logic_to_iron_and_coal_deposits/
 17. Hexagonal Grids - Red Blob Games,
<https://www.redblobgames.com/grids/hexagons/>
 18. Map Generation Fixing - Skymods - Game Mods Catalogue,
<https://catalogue.smods.ru/archives/118116>
 19. Better start locations? Too much tundra | CivFanatics Forums,
<https://forums.civfanatics.com/threads/better-start-locations-too-much-tundra.540004/>